Code-Guided Video Editing with Source-Grounded Executable Proxies

Yuanhong Zheng
Peking University

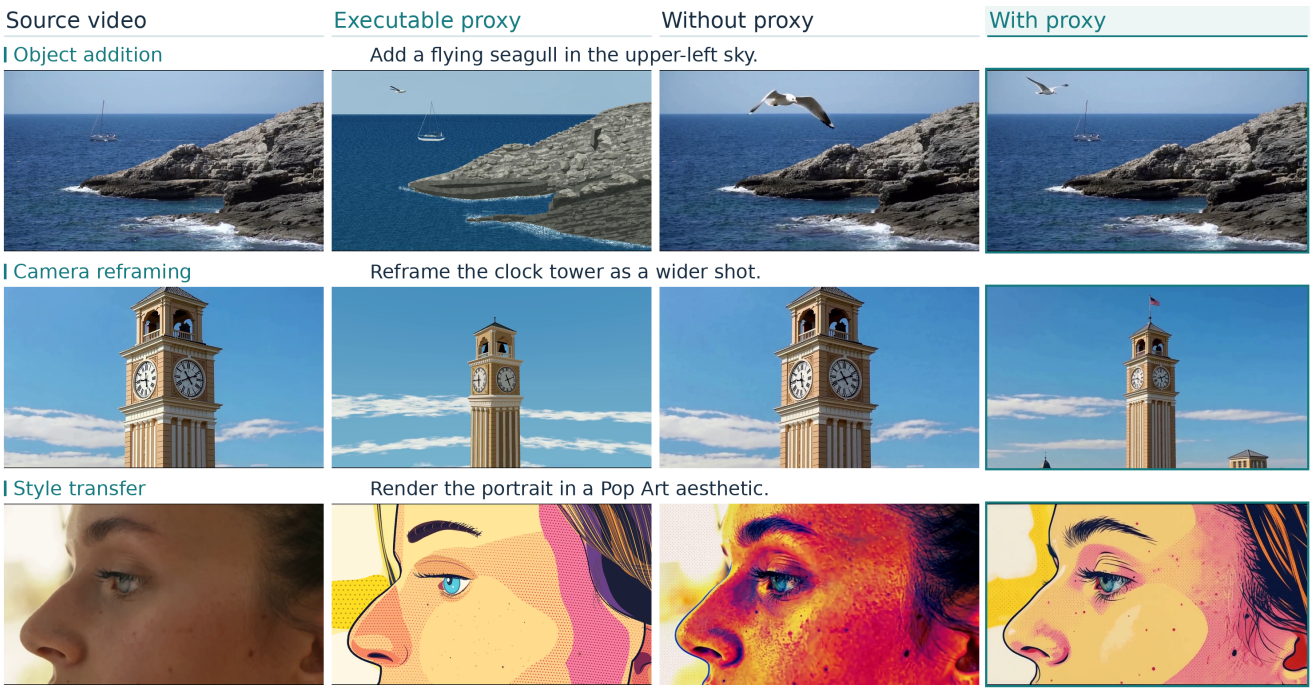

Figure 1. **Executable proxies for source-video editing.** Rows show object addition, camera reframing, and Pop Art stylization. Each row compares the source, rendered proxy, and matched outputs without and with early-step proxy guidance. In these examples, guidance improves object scale, realizes the wider composition, and produces the requested graphic treatment. All views within a row are time-aligned.

Abstract

Instruction-guided video editing requires coordinated scene changes while preserving source content. We express an edit through an executable scene program whose render provides a visual proxy for video synthesis. Our formulation comprises Video2Code *for source-scene estimation,* next-code prediction *for its edited successor, and* Code2Video *for video realization. A training-free prototype jointly implements the first two stages with a video-aware coding agent. A frozen generator then uses the source throughout sampling and the edited proxy only in the early steps. On 365 completed OpenVE-Bench pairs spanning eight tasks, three Gemini 3.1 Pro (preview) judgments per output yield a category-macro score of 2.84 with proxy guidance, compared with 2.74 without it. The largest gains occur in text and subtitles, camera editing, and object addition. Diagnostic cases reveal a gap between specifying and realizing an edit: a change visible in the proxy can be omitted, partially followed, or accompanied by unwanted appearance transfer. These findings motivate learning to align executable edit specifications with video synthesis.*

1. Introduction

An instruction such as "add a bird in the upper-left sky" specifies more than a new object. It also implies a location, scale, trajectory, and relationship to the existing scene. Similarly, removing an object requires a plausible account of the revealed background, and widening a shot requires a target camera configuration. Each intervention must remain consistent with the source video's unedited content and timing. The editing task therefore couples two requirements: specifying the desired change and retaining the source information needed to realize it.

Diffusion models provide powerful priors for visual synthesis [10, 22]. Video editing methods use attention reuse, feature propagation, and reference conditioning to carry source information into edited outputs [7, 12, 14, 21]. These advances leave a complementary representation question: *how can the intended intervention be made explicit and inspected before the final video is generated?* A natural-language request can describe an edit, but its concrete spatial and temporal realization becomes apparent only after a model or another visual system instantiates it.

We use an executable scene program to instantiate that realization. The program specifies entities, attributes, camera configuration, and behavior over time; executing it produces a rendered proxy of the edited scene. The program is the editable representation, and the proxy video is its visual interface to the generator. Figure 1 illustrates this interface across object addition, camera reframing, and stylization. The source video supplies appearance and performance that the procedural scene approximates, while the proxy makes the requested intervention visible. This separation allows an agent or a user to inspect the editing plan independently of its final generative realization.

We formulate the process as *Video2Code* $\rightarrow$ *next-code prediction* $\rightarrow$ *Code2Video*. Video2Code establishes a procedural account of the source scene. Next-code prediction defines its successor under the editing instruction, and Code2Video renders that successor and uses its proxy to guide synthesis. Here, "next" denotes an edited program state, with a complete video timeline. Our training-free prototype implements source grounding and program editing jointly: one coding-agent session observes the source and instruction and emits the edited program directly. The three stages define distinct responsibilities, although the first two are fused in this implementation.

Our implementation builds on the executable previews and early-step conditioning of Code Video Model [15]. We adapt this approach to editing observed footage by grounding program construction in the source video and maintaining that video as a reference throughout generation. The edited proxy is supplied only during the first five of 20 sampling steps. This schedule lets the proxy influence early synthesis while the source remains available for subsequent refinement, without additional model training.

We evaluate 365 completed paired cases across the eight tasks of OpenVE-Bench [8]. With three Gemini 3.1 Pro (preview) judgments per output and equal task weighting, proxy guidance increases the mean score from 2.74 to 2.84. The improvements concentrate in text and subtitles, camera editing, and object addition, with mixed results elsewhere. Inspecting the intermediate representation also exposes failures that a final-video score alone cannot localize: the program may already express the edit, while synthesis omits it or transfers unwanted proxy appearance. These observations motivate learning at the proxy-to-video interface.

Our contributions are:

- An executable representation of source-video editing as scene reconstruction, instruction-conditioned program transition, and source-grounded video realization.
- A training-free implementation that jointly grounds and edits a scene program, then combines its rendered proxy with a persistent source reference under early-step conditioning.
- A paired study of 365 completed OpenVE-Bench cases, with repeated judgments, equal-weight category aggregation, and visual diagnostics separating edit specification from its generative realization.

2. Related Work

Source preservation in video editing. Tune-A-Video adapts an image diffusion model to a source video through one-shot tuning [26]. FateZero and TokenFlow preserve source information through attention reuse and propagation of diffusion features, respectively [7, 21]. FLATTEN introduces optical-flow-guided attention, while RAVE combines frame interactions with randomized noise shuffling [4, 13]. CoDeF represents canonical content and temporal deformation to propagate image edits through a video [20]. AnyV2V combines an edited first frame with feature injection into an image-to-video model [14]. These approaches inform how source information can persist during editing. We study an additional interface: an executable target-scene program whose render communicates the requested intervention to a video generator.

Structured conditions for visual synthesis. Structure- and content-guided generation separates conditioning signals for these two aspects of video synthesis [5]. ControlNet learns spatial control from conditions such as edges, depth, and pose [29], while VideoComposer combines text, spatial constraints, and temporal conditions for video synthesis [24]. MotionCtrl and CameraCtrl provide explicit interfaces for object or camera motion [9, 25]. ReVideo combines first-frame content editing with trajectory-based motion control [18], and VACE unifies references, video

inputs, and masks for creation and editing [12]. Our rendered proxy encodes object layout, camera configuration, motion, and semantic changes in one visual condition. We use the reference-conditioned MiniMax-H3 model for video realization [17].

Executable visual representations. Holodeck generates three-dimensional environments from language and spatial constraints [28]; BlenderAlchemy uses visual feedback to search over graphics-editing actions [11]. The Scene Language combines programs, semantic descriptions, and identity embeddings [30], and Code-as-World studies executable representations for physical reasoning [23]. Code2Video generates educational animations through executable code [2]. Code2Video-Compass evaluates code-based generation from text, image, and video inputs [3], alongside recent open-model efforts in code-driven video creation [19]. These works motivate programs as representations that can be executed, inspected, and revised.

Code Video Model is the closest foundation for our implementation [15]. It renders programs into video previews, guides synthesis with a proxy and an appearance reference, and introduces early-step conditioning to limit transfer of the proxy's rendered appearance. Its applications also include scene editing. We reuse this mechanism in an observed-video setting: source footage grounds program construction and remains a persistent reference during synthesis. The Video2Code–next-code–Code2Video formulation organizes this source estimation, intervention, and realization interface.

Editing data and evaluation. InstructPix2Pix learns instruction-based image editing from generated pairs [1]. InsViE-1M constructs and filters video-editing triplets for training an instruction-following editor [27]. OpenVE-3M provides editing data and OpenVE-Bench, which spans eight editing tasks [8]. OmniEdit-Bench and VEFX-Bench further examine video-specific coverage, instruction fidelity, and content preservation [6, 16]. We adopt OpenVE-Bench's task-specific rubrics, average repeated judgments, and report equal-weight category aggregation. Executable intermediates additionally allow us to inspect whether a failure arises before or after the intended edit is expressed in the proxy.

3. Method

3.1. Editing as a program transition

Let $V_{\mathrm{src}} = \{I_i\}_{i=0}^{N-1}$ denote a source video and e an editing instruction. We describe editing through a source scene program P_0, its edited successor P_1, and a video realization:

$$P_0 = \mathcal{A}_{\mathrm{rec}}(V_{\mathrm{src}}), \tag{1}$$

$$P_1 = \mathcal{A}_{\mathrm{next}}(P_0, e; V_{\mathrm{src}}), \tag{2}$$

$$V_{\mathrm{proxy}} = \mathcal{R}(P_1), \tag{3}$$

$$V_{\mathrm{edit}} = \mathcal{G}(V_{\mathrm{src}}, e, V_{\mathrm{proxy}}). \tag{4}$$

Here $\mathcal{A}_{\mathrm{rec}}$ estimates scene structure, $\mathcal{A}_{\mathrm{next}}$ specifies an instruction-conditioned program transition, $\mathcal{R}$ executes and renders a program, and $\mathcal{G}$ is a frozen reference-conditioned video model. Each program describes a scene over the full video timeline; P_0 and P_1 index editing states. The rendered proxy is an observable consequence of the edited program.

Joint implementation. Equations (1)–(4) define the logical interfaces. Our prototype implements the first two jointly as $P_1 = \mathcal{A}_{\mathrm{joint}}(V_{\mathrm{src}}, e)$: a multimodal coding agent receives the source observations and instruction and directly produces the edited program. Only P_1 is retained; the experiments do not materialize or independently evaluate P_0. Figure 2 distinguishes this joint implementation from the three-stage formulation.

3.2. Video2Code: grounding the source scene

Source grounding identifies the scene structure on which the requested edit acts: visible entities, their relative layout and occlusions, the camera, and motion over time. The target is a procedural account sufficient to express the intervention. Unique three-dimensional geometry and complete photographic reconstruction are unnecessary for this role.

The coding agent inspects the source video through time-indexed frames and constructs a Three.js scene. Object groups and geometry represent entities; camera parameters determine the view; time-dependent transforms encode motion and event timing. Materials, lighting, and procedural effects convey semantic attributes. The source composition, viewing direction, action timing, cuts, and occlusion relationships anchor the program wherever the edit leaves them unchanged. These constraints apply to the joint source-grounding and editing session.

The task contract requires a fully procedural render. Source imagery is used for observation and is excluded from rendered textures, video backgrounds, plates, and composited overlays. Consequently, the proxy exposes a constructed scene description rather than a copy of the input pixels. The source video remains available downstream for appearance and performance that the procedural approximation does not capture.

3.3. Next-code prediction: specifying the intervention

The program transition expresses the edit through changes to an executable scene. Additions introduce entities with ex-

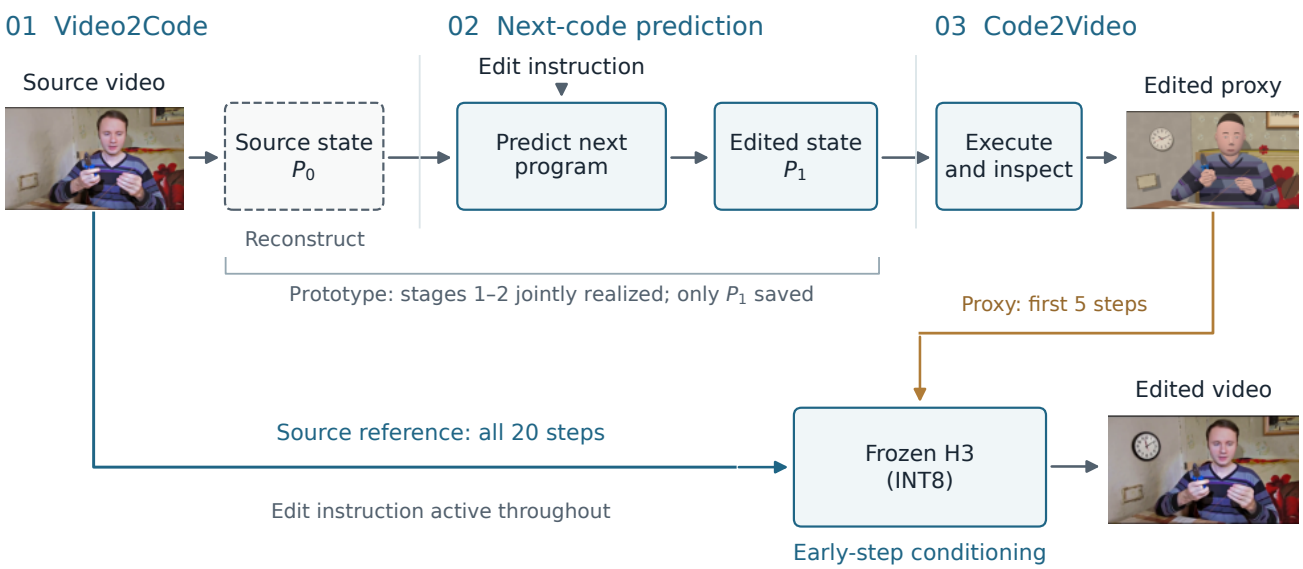

Figure 2. **Editing through executable scene programs.** Video2Code estimates the source scene, next-code prediction specifies its edited successor, and Code2Video renders the edited program to guide video synthesis. The training-free implementation jointly performs the first two stages in one coding-agent session and retains the edited program. The frozen generator uses the source and edit instruction for all 20 steps, with proxy conditioning restricted to the first five steps.

plicit placement, scale, and behavior. Removals omit the target entity and represent the revealed context. Camera edits change the viewing configuration; stylization and creative edits modify materials, geometry, or procedural effects. The successor program retains scene relationships that remain relevant to the instruction and can rewrite code as needed.

For example, the wall-clock scene in Fig. 2 grounds the person, wall, and action timing in the source, then introduces the clock's geometry, placement, and hand animation. The program exposes both the edited scene and the parameters that determine its render. In the joint implementation, these source-grounding and intervention operations are performed within one session.

Deterministic execution and visual refinement. Every scene exposes a `seek(t)` interface that updates camera, transforms, visibility, materials, and animation at time t. The state at a timestamp must be independent of the order in which timestamps are visited. The agent renders keyframes and contact sheets, compares them with the source observations and edit request, and revises the program. The refinement contract covers composition, geometry and occlusion, motion and timing, and semantic legibility. The encoded proxy is inspected again before use. Execution checks validate resolution, frame count, silent output, and per-frame camera metadata; these establish an executable condition without guaranteeing semantic correctness.

3.4. Code2Video: realizing the edited scene

Rendering P_1 at the normalized source timestamps gives $V_{\text{proxy}} = \{\mathcal{R}(P_1, t_i)\}_{i=0}^{N-1}$. We provide this proxy and the source as distinct video references to MiniMax-H3 [17]. The prompt identifies `<Video 1>` as the source and `<Video 2>` as the edited structural proxy. The source anchors retained appearance and performance, while the proxy communicates the requested intervention. Appendix B provides the reference-role instructions.

We normalize the source to 960×544 at 24 frames per second using proportional resizing and padding, and render the proxy on the same timeline. Any extra frames required for model alignment hold the final state of both conditions. Outputs are trimmed to the source's visible interval.

Early-step proxy conditioning. Following Code Video Model [15], we restrict proxy conditioning to the initial sampling steps. For zero-based step k, total steps K, and proxy budget K_e,

$$c_k = \begin{cases} \text{Enc}(e, V_{\text{src}}, V_{\text{proxy}}), & 0 \le k < K_e, \\ \text{Enc}(e, V_{\text{src}}), & K_e \le k < K. \end{cases} \tag{5}$$

Enc includes multimodal semantic conditioning and variational autoencoder (VAE) reference conditioning. We set $K = 20$ and $K_e = 5$ and switch both pathways from source-plus-proxy to source-only after step five. The source

VAE reference latent is shared across conditioning states; the edit instruction remains active throughout. Sampling follows one continuous latent trajectory, without a restart or re-noising at the switch. All generator and encoder parameters remain frozen.

4. Experiments

4.1. Experimental setup

Benchmark and coverage. OpenVE-Bench contains 431 source-video and edit-request pairs across eight tasks [8]. We evaluate the 365 pairs for which proxy construction, validation, and both output generations have completed. The remaining cases comprise 14 moderation skips and 52 unresolved proxy generation or validation cases. Eligibility is determined by pipeline completion, and every completed pair is included in the quantitative comparison. The results therefore describe the completed subset.

Matched generation. Both groups use the same normalized source, main edit request, frozen quantized MiniMax-H3, seed 42, and 20 Euler steps. The source remains a reference for all steps. The guided group additionally receives the edited proxy for the first five steps; the baseline receives no proxy. Appendices A and B specify the implementation and reference-role prompts.

Repeated judgments and aggregation. We use the official task-specific rubrics and per-response score parser, with Gemini 3.1 Pro (preview) at temperature 0.7 as the evaluator. Each judgment receives the original source video, generated output, and edit request. The proxy is excluded from evaluation inputs. For each output, we average three valid judgments, each an official mean of three rubric dimensions on a 1–5 scale. The primary metric gives equal weight to the eight category means. The secondary micro average gives equal weight to cases. Appendix D details video input, repeated calls, and rounding.

4.2. Paired benchmark comparison

As shown in Table 1, proxy guidance increases the primary category-macro score from 2.74 to 2.84, a gain of 0.10 points under the selected evaluator. The largest gains occur in text and subtitles (+0.56), camera editing (+0.45), and object addition (+0.28). These improvements are consistent with the role of an executable preview in making graphical changes, target framing, and object placement explicit. Figures 1 and 3 illustrate corresponding edits; Appendix C provides successful examples from all eight categories.

Performance varies across tasks. Background editing improves by 0.03, and local change is unchanged at the displayed precision. Global style, creative editing, and object removal decrease by 0.05, 0.14, and 0.34, respectively. The overall gain thus coexists with category-specific regressions, which motivate examining the intermediate proxy alongside the final output.

Table 1. **OpenVE-Bench paired evaluation.** Gemini 3.1 Pro (preview), three ratings per output, official task rubrics; 1–5, higher is better. Both groups use the same 365 cases. The proxy participates for the first 5 of 20 steps. The primary metric weights all eight categories equally. Δ subtracts the displayed arm means.

Task	N	No proxy	+ Proxy	Δ
Global style	55	2.43	2.38	−0.05
Local change	57	3.39	3.39	0.00
Background change	46	2.39	2.42	+0.03
Object removal	49	2.40	2.06	−0.34
Object addition	60	2.40	2.68	+0.28
Creative edit	26	3.47	3.33	−0.14
Camera edit	39	2.44	2.89	+0.45
Text and subtitles	33	3.00	3.56	+0.56
Category macro	365	**2.74**	**2.84**	**+0.10**
Sample micro	365	2.69	2.78	+0.09

4.3. Diagnosing proxy-to-video failures

An explicit intermediate helps distinguish failure to express an edit from failure to realize an expressed edit. Figure 4 shows selected examples from the same evaluated run. Each proxy visibly expresses the relevant intervention, although the final output does not faithfully realize it.

Edit omission. The bird-removal proxy omits the target and supplies a reconstructed background, yet the guided video retains the bird; the baseline removes it. For mountain stylization, the proxy contains monochrome drawing and hatching, while the guided output largely retains a photographic appearance. In both cases, the requested change is legible before synthesis but is missing from the generated video.

Partial realization. The creative-edit proxy depicts glowing jellyfish-like forms in place of snowflakes. The guided video introduces jellyfish but retains visible snowflakes alongside them. The generator follows part of the intervention while preserving source elements that the request intended to replace.

Appearance leakage. The suit-replacement proxy contains the requested gray suit and red tie. The guided video follows the clothing edit but also adopts the proxy's cartoon appearance, whereas the baseline retains a more photographic style. Restricting proxy guidance to five steps therefore does not eliminate unwanted appearance transfer in every case.

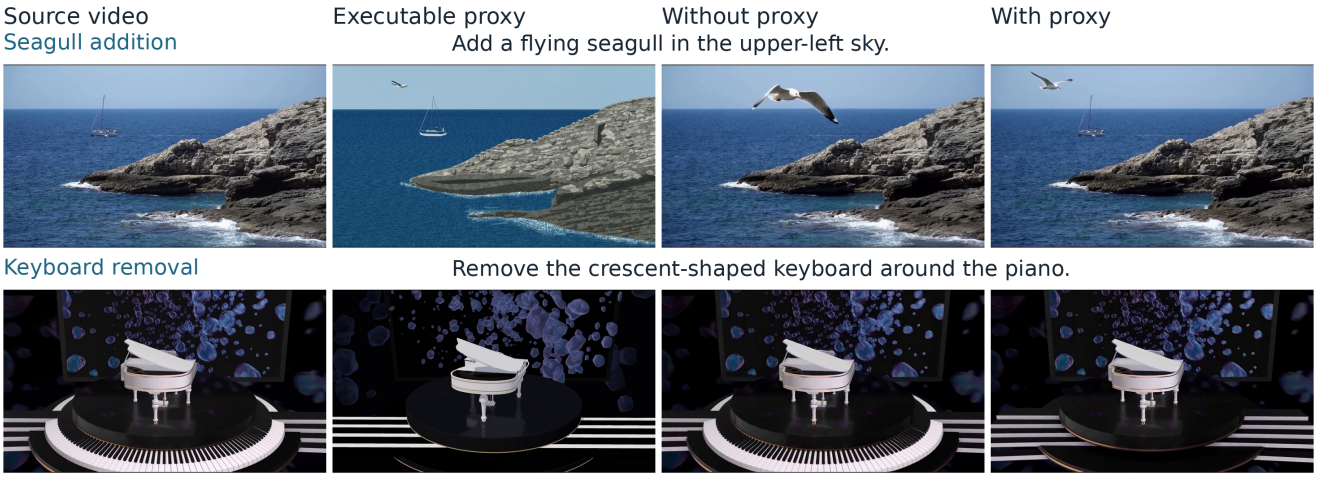

Figure 3. **Successful spatial edits.** Top: the guided seagull follows the proxy's scale and placement more closely; the baseline produces a much larger foreground bird. Bottom: the guided output removes the crescent-shaped keyboard around the piano, while the baseline retains it. The source, proxy, and both outputs are time-aligned within each row.

These observations identify concrete failures at the proxy-to-video interface. A legible edit specification can coexist with an unsuccessful final result, through either insufficient adherence or excessive transfer of approximate proxy appearance. The selected cases characterize these behaviors; their prevalence and underlying causes require further study.

5. Discussion and Limitations

Learning to follow an executable edit. The results distinguish the capacity to specify an intervention from the capacity to synthesize it faithfully. In Fig. 4, the desired change is already visible in the proxy, but the generator can omit it or inherit unwanted rendering traits. The pretrained H3 model remains frozen throughout our study and receives no adaptation to these source–edited-proxy pairs. A natural next step is to learn how to use the proxy's intended intervention while retaining appearance and performance from the source. Learned spatial control provides a precedent for exploiting explicit conditions [29]. Whether proxy-aware adaptation resolves the observed video-editing failures must be established through training and held-out evaluation; the present diagnostics do not isolate the cause of every regression.

Structured editing data. The prototype records tuples $(V_{\text{src}}, e, P_1, V_{\text{proxy}}, V_{\text{edit}})$, linking an instruction, an executable edited scene, and its generative realization. Materializing and validating the source program would extend these to $(V_{\text{src}}, e, P_0, P_1, V_{\text{proxy}}, V_{\text{edit}})$ and expose the program transition itself. Such records could support learning video-to-program reconstruction, instruction-conditioned program editing, and proxy-conditioned synthesis. Programs also provide object, camera, and temporal metadata alongside their renders. Training targets would require independent quality screening: a successful render does not ensure a successful edit, and generated targets remain pseudo-labels until validated. The importance of data construction and filtering is established in instruction-based image and video editing [1, 8, 27].

Scope of the evidence. A procedural proxy approximates scene structure and may miss source details or behavior. Its execution checks do not establish semantic correctness. The quantitative comparison covers 365 completed pairs from a 431-case benchmark, so unresolved construction and moderation can introduce selection bias. It also uses one video generator, one generation seed, and one automated evaluator. Repeating judgments reduces dependence on an individual response but does not measure variation across generated samples or establish statistical significance. The selected visual examples explain observable behavior rather than its frequency. Further evaluation should complete benchmark coverage, include multiple seeds and human judgments, and measure program-construction cost. Editing-specific protocols support separating instruction fidelity, preservation, realism, and consistency [6, 16]. Evaluating source reconstruction, program editing, and video realization separately would further clarify where improvements are needed.

6. Conclusion

We studied executable scene programs as an intermediate representation for source-video editing. The

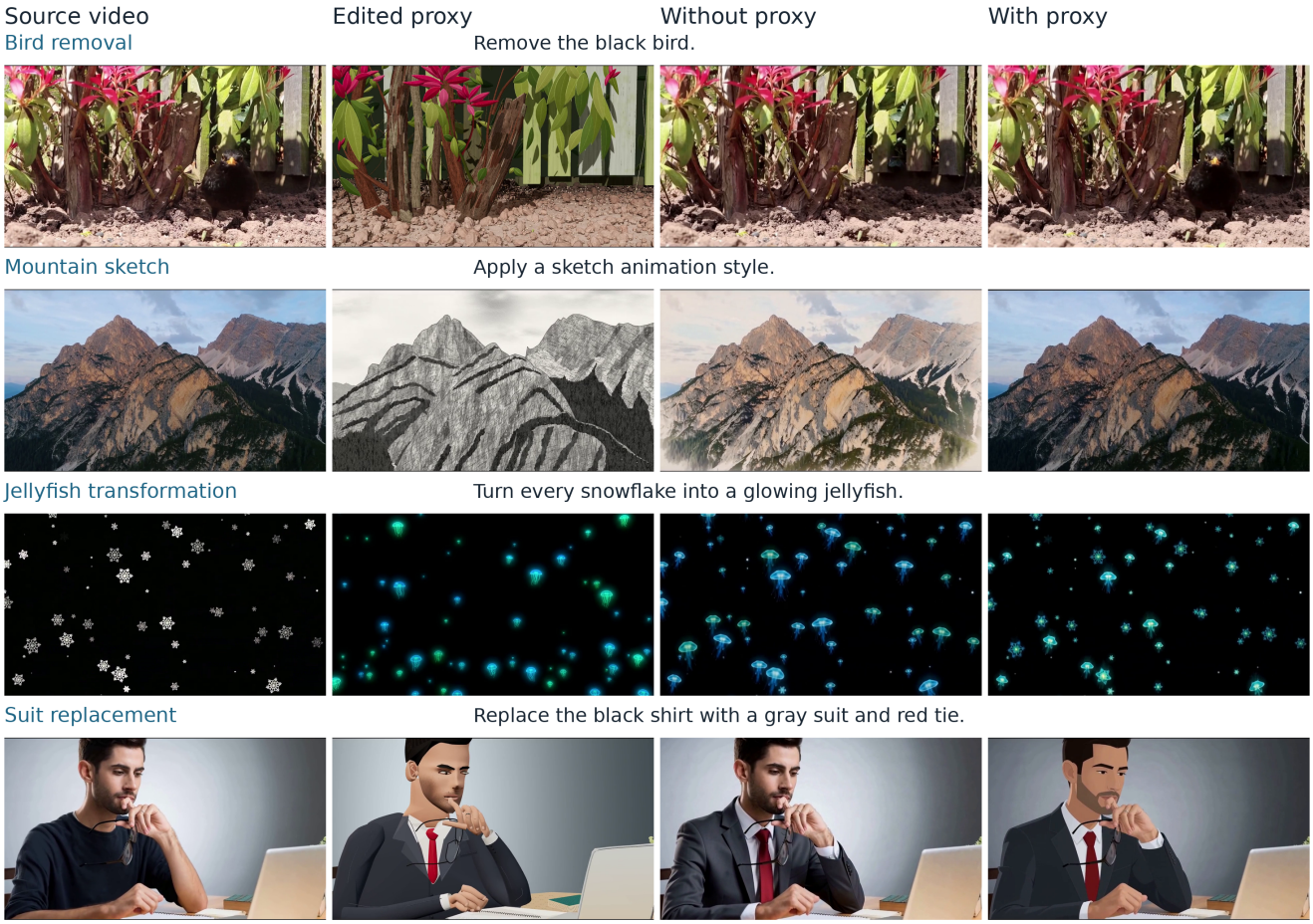

Figure 4. **A gap between edit specification and realization.** From top to bottom: the proxy omits the bird but the guided output retains it; the mountain proxy's sketch treatment is largely absent from the generated result; jellyfish coexist with retained snowflakes; and the requested suit appears together with unwanted cartoon styling. All examples use the evaluated run in Table 1. Appendix Fig. 9 provides additional timestamps.

Video2Code–next-code–Code2Video formulation separates source grounding, intervention specification, and video realization. A training-free prototype jointly grounds and edits a program, then uses its rendered proxy to condition a frozen generator alongside the source. On 365 completed OpenVE-Bench pairs, repeated Gemini 3.1 Pro (preview) judgments yield a 0.10-point category-macro gain, with the largest gains in text and subtitles, camera editing, and object addition. Diagnostic cases show that a change expressed by the program can still be omitted, partially realized, or accompanied by unwanted appearance transfer. This gap motivates learning the interface between executable edit specifications and video synthesis.

References

[1] Tim Brooks, Aleksander Holynski, and Alexei A. Efros. InstructPix2Pix: Learning to Follow Image Editing Instructions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18392–18402, 2023.

[2] Yanzhe Chen, Kevin Qinghong Lin, and Mike Zheng Shou. Code2Video: A Code-centric Paradigm for Educational Video Generation. *arXiv preprint arXiv:2510.01174*, 2025.

[3] Zhenghan Chen, Zekai Shao, Xin Lin, Leting Cheng, and Siming Chen. Code2Video-Compass: A Comprehensive Benchmark for General-Purpose Code-Based Video Generation across T2V, I2V, and V2V. *arXiv preprint arXiv:2610.28416*, 2026.

[4] Yuren Cong, Mengmeng Xu, Christian Simon, Shoufa Chen, Jiawei Ren, Yanping Xie, Juan-Manuel Perez-Rua, Bodo Rosenhahn, Tao Xiang, and Sen He. FLATTEN: optical FLow-guided ATTENtion for consistent text-to-video editing. In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Patrick Esser, Johnathan Chiu, Parmida Atighehchian, Jonathan Granskog, and Anastasis Germanidis. Structure and Content-Guided Video Synthesis with Diffusion Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7346–7356, 2023.

[6] Xiangbo Gao, Sicong Jiang, Bangya Liu, Xinghao Chen, Minglai Yang, Siyuan Yang, Mingyang Wu, Jiongze Yu, Qi Zheng, Haozhi Wang, Jiayi Zhang, Jie Yang, Zihan Wang, Qing Yin, and Zhengzhong Tu. VEFX-Bench: A Holistic Benchmark for Generic Video Editing and Visual Effects. *arXiv preprint arXiv:2604.16272*, 2026.

[7] Michal Geyer, Omer Bar-Tal, Shai Bagon, and Tali Dekel. TokenFlow: Consistent Diffusion Features for Consistent Video Editing. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Haoyang He, Jie Wang, Jiangning Zhang, Zhucun Xue, Xingyuan Bu, Qiangpeng Yang, Shilei Wen, and Lei Xie. OpenVE-3M: A Large-Scale High-Quality Dataset for Instruction-Guided Video Editing. *arXiv preprint arXiv:2512.07826*, 2025.

[9] Hao He, Yinghao Xu, Yuwei Guo, Gordon Wetzstein, Bo Dai, Hongsheng Li, and Ceyuan Yang. CameraCtrl: Enabling Camera Control for Video Diffusion Models. In *International Conference on Learning Representations (ICLR)*, 2025.

[10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020.

[11] Ian Huang, Guandao Yang, and Leonidas Guibas. BlenderAlchemy: Editing 3D Graphics with Vision-Language Models. In *European Conference on Computer Vision (ECCV)*, 2024.

[12] Zeyinzi Jiang, Zhen Han, Chaojie Mao, Jingfeng Zhang, Yulin Pan, and Yu Liu. VACE: All-in-One Video Creation and Editing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17191–17202, 2025.

[13] Ozgur Kara, Bariscan Kurtkaya, Hidir Yesiltepe, James M. Rehg, and Pinar Yanardag. RAVE: Randomized Noise Shuffling for Fast and Consistent Video Editing with Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6507–6516, 2024.

[14] Max Ku, Cong Wei, Weiming Ren, Harry Yang, and Wenhu Chen. AnyV2V: A Tuning-Free Framework For Any Video-to-Video Editing Tasks. *Transactions on Machine Learning Research*, 2024.

[15] Zhiyang Liang, Jiawei Zhang, Guocheng Qian, Dongdong Chen, and Ziyu Wan. Code Video Model, 2026. Technical report.

[16] Chenxuan Miao, Yutong Feng, Yi Lu, Yunfeng Yan, Donglian Qi, Shiwei Zhang, Yu Liu, Xi Chen, and Hengshuang Zhao. OmniEdit-Bench: A Comprehensive Benchmark for Instruction-based Video Editing. *arXiv preprint arXiv:2608.05049*, 2026.

[17] MiniMax. MiniMax-H3. Official model and code release, 2026.

[18] Chong Mou, Mingdeng Cao, Xintao Wang, Zhaoyang Zhang, Ying Shan, and Jian Zhang. ReVideo: Remake a Video with Motion and Content Control. In *Advances in Neural Information Processing Systems*, 2024.

[19] OpenSenseNova. SenseNova 6.8 Code2Video. Official model and code release, 2026.

[20] Hao Ouyang, Qiuyu Wang, Yuxi Xiao, Qingyan Bai, Juntao Zhang, Kecheng Zheng, Xiaowei Zhou, Qifeng Chen, and Yujun Shen. CoDeF: Content Deformation Fields for Temporally Consistent Video Processing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8089–8099, 2024.

[21] Chenyang Qi, Xiaodong Cun, Yong Zhang, Chenyang Lei, Xintao Wang, Ying Shan, and Qifeng Chen. FateZero: Fusing Attentions for Zero-shot Text-based Video Editing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15932–15942, 2023.

[22] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022.

[23] Hanyang Wang, Yimo Cai, Weiliang Chen, Jiawei Chi, Haowen Sun, Qiyu Dai, Yi-Hsin Hung, Xingzhuo Guo, Jinshan Ren, Runmao Yao, Ziwei Liu, Mingsheng Long, Yueqi Duan, Jun Gao, Jiangran Lyu, Fangfu Liu, and Jialong Wu. Code as Worlds: Agentic Discovery of Executable World Representations for Physical Reasoning. *arXiv preprint arXiv:2608.27549*, 2026.

[24] Xiang Wang, Hangjie Yuan, Shiwei Zhang, Dayou Chen, Jiuniu Wang, Yingya Zhang, Yujun Shen, Deli Zhao, and Jingren Zhou. VideoComposer: Compositional Video Synthesis with Motion Controllability. In *Advances in Neural Information Processing Systems*, 2023.

[25] Zhouxia Wang, Ziyang Yuan, Xintao Wang, Tianshui Chen, Menghan Xia, Ping Luo, and Ying Shan. MotionCtrl: A Unified and Flexible Motion Controller for Video Generation. In *ACM SIGGRAPH Conference Papers*, 2024.

[26] Jay Zhangjie Wu, Yixiao Ge, Xintao Wang, Stan Weixian Lei, Yuchao Gu, Yufei Shi, Wynne Hsu, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Tune-A-Video: One-Shot Tuning of Image Diffusion Models for Text-to-Video Generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7623–7633, 2023.

[27] Yuhui Wu, Liyi Chen, Ruibin Li, Shihao Wang, Chenxi Xie, and Lei Zhang. InsViE-1M: Effective Instruction-based Video Editing with Elaborate Dataset Construction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16692–16701, 2025.

[28] Yue Yang, Fan-Yun Sun, Luca Weihs, Eli VanderBilt, Alvaro Herrasti, Winson Han, Jiajun Wu, Nick Haber, Ranjay Krishna, Lingjie Liu, Chris Callison-Burch, Mark Yatskar, Aniruddha Kembhavi, and Christopher Clark. Holodeck: Language Guided Generation of 3D Embodied AI Environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16227–16237, 2024.

[29] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding Conditional Control to Text-to-Image Diffusion Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3836–3847, 2023.

[30] Yunzhi Zhang, Zizhang Li, Matt Zhou, Shangzhe Wu, and Jiajun Wu. The Scene Language: Representing Scenes with Programs, Words, and Embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24625–24634, 2025.

A. Implementation and Figure Provenance

Program construction. Our implementation adapts the Code Video Model program-and-render workflow to observed source videos. The three-stage formulation separates source reconstruction from prediction of the edited program at the representation level. The existing prototype gives the source and instruction jointly to the agent and serializes only P_1, together with its rendered proxy. It does not run two independently checkpointed coding modules or save a standalone pre-edit P_0 reconstruction. The coding stage uses the Codex CLI with GPT-6 Astra and the `xhigh` reasoning setting. Each case has its own procedural Three.js workspace, source-frame index, rendering helper, and task contract. The agent renders and directly inspects keyframes and encoded-video contact sheets. The renderer evaluates frame i at time $i/24$ through a deterministic `window.reconstruction.seek(t)` interface. A camera-state accessor exports position, quaternion, and field of view after the corresponding time has been evaluated. This metadata is used to validate the procedural output. It is not supplied as a separate camera-control tensor to the video model.

Video synthesis. We use the INT8 H3 implementation from the existing early-step experiment, with a quantized Qwen visual-language encoder. Video-VAE and audio-VAE computation use the established FP16 and FP32 paths, respectively. Sampling uses 20 Euler steps, seed 42, video/audio shifts of 12/3, and the unchanged sigma schedule of that experiment. Both multimodal semantic conditioning and VAE reference conditioning switch from source-plus-proxy to source-only after the fifth step. There is one continuing audiovisual latent trajectory. The source reference latent is shared across the full and reduced conditioning states. All parameters remain frozen.

For throughput, the existing experiment used eight resident single-GPU workers to generate independent cases. This is case-level parallelism; one video does not require the eight workers to participate in a single distributed forward pass. Figure construction only decodes existing videos and does not run H3 again.

Timeline and audio. Source videos are normalized to 24 FPS and proportionally resized and padded to 960×544. If the normalized source has N frames, alignment uses the smallest admissible $N' \geq N$ satisfying $N' = 17m+5$. The additional $N' - N$ frames hold the final source and proxy states. The displayed edited videos are trimmed to the original N-frame interval. Proxies are silent. Generated audio is retained in the original experiment outputs; the present figures and qualitative descriptions concern visual editing.

Showcase selection. The figures use outputs from the `openve_h3_int8_early_step_20261031` experiment. Existing automated judgments were used to shortlist candidates, followed by direct visual inspection of the source, proxy, and both output groups. Selection targets successful illustrations of the requested edits. It is not a random benchmark sample. The successful gallery illustrates seagull addition, crescent keyboard removal, person aging, newsroom replacement, clock tower reframing, a celestial portal, Pop Art stylization, and subtitle removal. The method diagram uses wall clock addition. The same generated output is used wherever a case appears more than once.

Image construction. The supplied figure script decodes exact frame indices from the published source, proxy, and both generated MP4s. It preserves the full normalized frame and aspect ratio, then arranges the images with external labels. No image-generation model, retouching, or replacement of visual content is used in making the figures. The accompanying `research/showcase_provenance.json` records video hashes, sampled frame indices, timestamps, prompt hashes, and figure hashes. Figure edit summaries are paraphrases for readability; they are not additional instructions supplied during generation. The benchmark table and diagnostic figures are audited separately in `research/openve_repeat3_results.json` and `research/diagnostic_provenance.json`.

B. Conditioning Instructions

Coding-agent task contract. The task provides the source video, timestamped observations, the benchmark edit request, and exact output metadata. Its core requirements are to build a deterministic, fully procedural, editable Three.js scene; implement the requested edit; and retain unedited composition, geometry, occlusion, motion, cuts, and timing. Source imagery is available for observation only. External visual assets and source imagery are not used as rendered textures, plates, overlays, or masks. The contract also requires visual refinement passes covering composition, geometry and occlusion, motion and timing, and semantic legibility. The final encoded video is inspected before it is accepted as a condition.

Reference roles in H3. The no-proxy comparison uses the same source and main editing request, with a reference-role block that identifies only the source video. The proxy-guided group adds the second reference as described below. The original edit request is followed by the source-reference guidance below. This is the reference-role block used in the existing proxy-conditioned showcase runs; it introduces no shot-by-shot target-video description. The numbered video

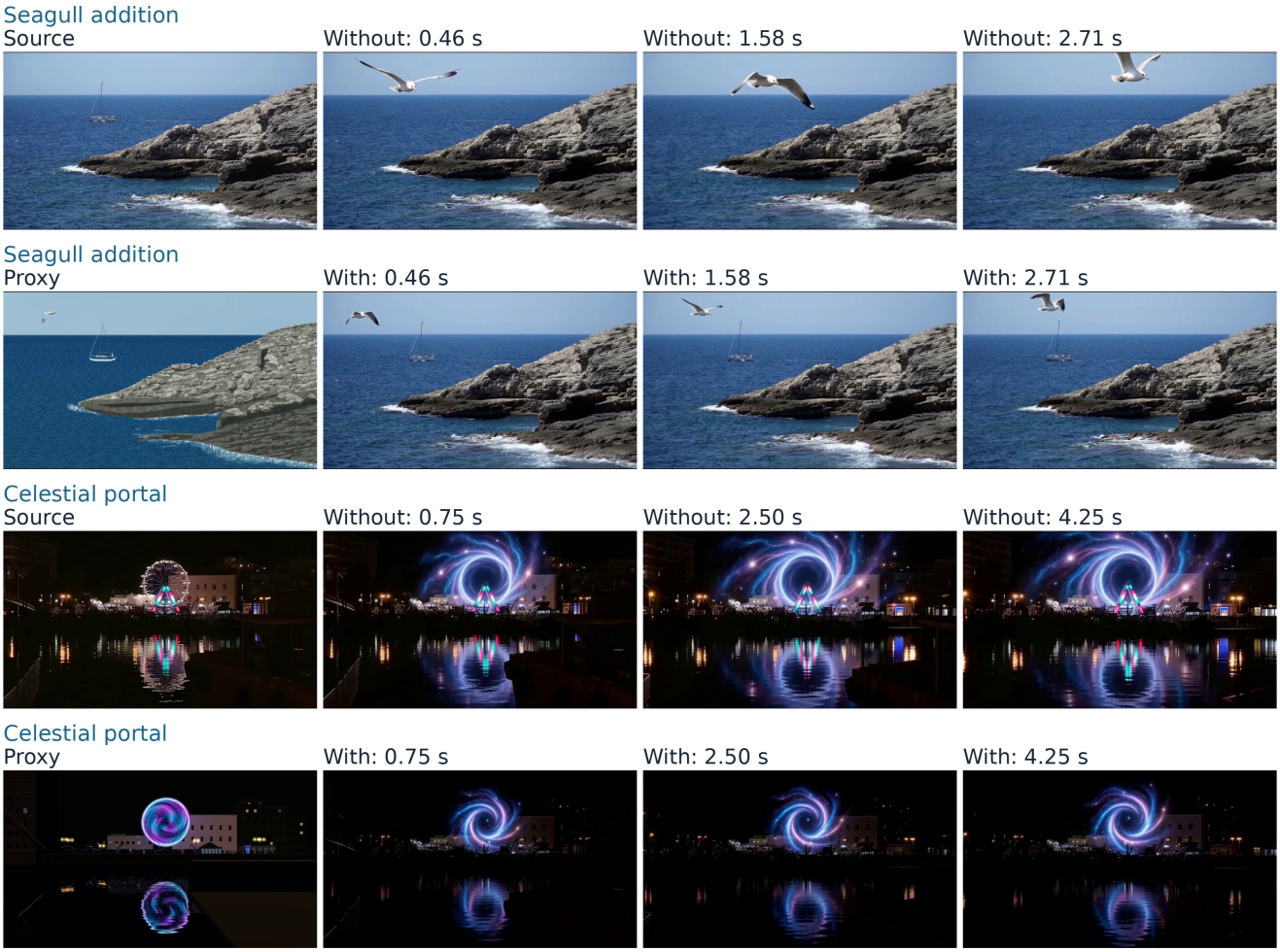

Figure 5. **Additional temporal views of existing showcase videos.** Each case has two rows: a source frame followed by three no-proxy timestamps, and a proxy frame followed by the matching guided timestamps. Top pair: seagull addition. Bottom pair: the celestial portal. All outputs are the same videos used in the gallery.

references correspond to their actual input order.

> `source_reference_guidance:` `<Video 1>` is the source video to edit. Preserve its object identities, appearance, color palette, lighting, textures, material treatment, rendering medium, and overall aesthetic, as well as its composition, poses, spatial layout, motion, camera, shot transitions, and timing, except where the main edit prompt requires a change. `<Video 2>` is the structural proxy video showing the requested edit. Use `<Video 2>` as structural guidance for the requested edit; do not transfer its rendered appearance to the output. Follow the main edit prompt for the requested changes.

The edit request and reference-role text stay in the conditioning prompt throughout the trajectory. After step five, the proxy's visual reference and its associated multimodal conditioning are withdrawn as specified in Eq. (5). For requested stylization or appearance edits, the explicit exception in the preservation instruction allows the intended change. Source preservation is a conditioning objective rather than a pixelwise compositing constraint.

C. Additional Qualitative Results

C.1. Gallery protocol

The gallery uses the same training-free experiment as Table 1 and includes one successful example from each OpenVE-Bench editing task. The wall-clock addition example also appears in the method diagram. Candidates were shortlisted using existing scores and then inspected visually; the gallery is a selected capability illustration.

Each comparison shows the source, rendered proxy, no-proxy output, and early-step proxy-guided output at matching timestamps. Both groups share the normalized source,

main edit request, frozen model, seed 42, and 20 sampling steps. The source remains a reference throughout, with the proxy added for the first five steps in the guided group. All displayed frames preserve the complete normalized composition. Edit summaries are paraphrased for readability; the recorded generation prompts are unchanged.

C.2. Entity and spatial edits

Local addition. The seagull addition example places an animated bird in the upper-left sky. The procedural scene contains the coastline and boat as context, with the new bird placed above them. The output in Fig. 3 introduces the bird while retaining the recognizable rocky shore and sea. The clock example in Fig. 2 offers a complementary indoor addition: the proxy locates a new object relative to an existing person and wall. These examples show how an addition can be made explicit before synthesis. The no-proxy outputs introduce the requested objects but overshoot the planned scale or placement; the guided outputs follow the corresponding proxy plans more closely.

Local removal. The keyboard removal example targets the crescent-shaped keyboard surrounding a grand piano. The baseline retains this prominent keyboard ring. The proxy explicitly omits the ring while retaining the central piano, stage, and surrounding visual arrangement. The guided output follows this removal (Fig. 3), illustrating how the rendered target scene can make an omission concrete.

Local change. The person aging example replaces a woman with a silver-haired older woman while retaining her pose and position. Both output groups perform the age edit. The baseline additionally changes her white jacket into a dark dress and introduces a large necklace. The proxy-guided result retains the original outfit and phone action (Fig. 6). This comparison illustrates a more targeted intervention with fewer unrequested appearance changes.

Background change. The newsroom replacement example changes the background to a futuristic cyber command center. The baseline realizes the new environment but removes the foreground woman. The proxy includes both the speaker and the target surroundings, and the guided output retains her while changing the background. This example highlights the distinct roles of a requested background change and a foreground subject that should remain in the shot.

C.3. Camera and creative edits

Camera edit. The clock tower example requests a wide shot. The agent constructs the tower and surrounding context under the target framing. The no-proxy output remains close to the original medium view. In the guided output, the tower occupies less of the frame and more of the adjacent environment becomes visible (Fig. 7). The program makes the framing choice explicit through its camera and scene configuration. The requested wide composition applies to the edited shot; it is not an instruction to add a new reveal or a new cut.

Creative edit. The waterfront example transforms a Ferris wheel into a celestial portal. The proxy contains a luminous circular structure and its reflection, providing a spatially organized account of the transformation. The no-proxy result leaves elements of the wheel structure within the transformation. The guided result forms a coherent blue-violet portal in the waterfront setting with reflected light on the water. The example shows how procedural geometry and effects can communicate a creative target while the source supplies scene context.

C.4. Global style and subtitle edits

Global style. The portrait example requests a Pop Art treatment of a close-up profile. The proxy renders the face with graphic contours, flat colors, and halftone details. The baseline mainly applies saturated colors to photographic texture. The guided result adopts drawn contours and halftone details while keeping the profile composition recognizable (Fig. 8). Here, appearance changes are part of the instruction. The source-retention guidance is explicitly subordinate to those requested changes.

Subtitle edit. The aerial landscape example removes subtitles from the top of the frame. The no-proxy output retains the text band. The proxy depicts the scene without text, and the guided output removes the band while retaining the recognizable fields, mountains, and cloudy sky. The example uses the video generator for the final completion; no separate text-erasure or compositing operation is applied to the displayed output.

Temporal inspection. The examples are generated as videos rather than independent edited frames. Appendix Fig. 5 provides additional timestamps for the seagull and portal examples. These sequences expose the continuing edit and its relation to the original shot.

Figure 6. **Targeted edits with better source-content retention.** Top: both outputs age the woman, but the no-proxy result changes her clothing; proxy guidance retains the original white jacket. Bottom: the baseline generates an empty cyber command center, while the guided result retains the foreground speaker in the new environment.

Figure 7. **Reframing and creative transformation.** Top: the no-proxy output remains close to the source framing; the guided result realizes the requested wider shot of the clock tower. Bottom: the baseline blends a portal effect with the original wheel structure; the guided output forms a distinct celestial portal with a corresponding reflection.

Figure 8. **Stylization and subtitle removal.** Top: the portrait is rendered in a Pop Art aesthetic. The baseline mainly changes photographic colors, whereas the guided output adds graphic outlines and halftone detail. Bottom: the baseline retains the subtitle band, while proxy guidance removes it from the aerial landscape.

D. Evaluation and Diagnostic Details

Evaluator and video input. Table 1 uses the model identifier `google/gemini-3.1-pro-preview`, the official OpenVE-Bench task-specific rubric text and score parser, temperature 0.7, and a maximum output budget of 8,192 tokens. The judge receives the full original source MP4 and the generated output MP4 trimmed to the source's visible interval, together with the task rubric and original edit request. Videos are uploaded directly; evaluation does not use locally extracted contact sheets or selected frames. No explicit video-sampling FPS is requested from the API, so the service's internal temporal sampling is not controlled in this experiment. The 24 FPS normalization used for synthesis is distinct from this unspecified evaluator sampling rate. The proxy is not an input to the judge. The reported rubric scores assess visual editing; generated audio quality is not a separate metric.

Three judgments of each output. Each of 365 cases has two fixed generated outputs and three valid judgments per output, giving 2,190 successful scoring responses. An existing valid response is reused as the first judgment when available (729 responses in total); the remaining slots use additional API calls with the same video pair, model, instruction, and rubric. Failed responses remain in the run records and are excluded from score means. No best-of-three selection is performed. The three judgments concern the same generated video, not three different video samples.

Aggregation. Let $d^a_{i,r,j}$ be rubric dimension j of judgment r for case i and arm a, with $j, r \in \{1, 2, 3\}$. We retain the official parser's per-response rounding,

$$s^a_{i,r} = \mathrm{round}_2\left(\frac{1}{3}\sum_{j=1}^{3} d^a_{i,r,j}\right), \qquad \bar{s}^a_i = \frac{1}{3}\sum_{r=1}^{3} s^a_{i,r}. \quad (6)$$

For the common paired case set I_c of task c, the primary metric is

$$S^a_{\mathrm{macro}} = \frac{1}{8}\sum_{c=1}^{8}\frac{1}{|I_c|}\sum_{i\in I_c}\bar{s}^a_i. \quad (7)$$

The secondary micro score gives every case equal weight across the 365 pairs. We retain precision through repeat, task, and macro averaging, and round the displayed means to two decimals. The table's displayed differences subtract these displayed arm means, matching the existing visualization. All eight tasks have paired coverage, but their case counts differ. The 14 moderation skips and 52 unresolved program-generation or validation cases are outside this paired subset; no scores are imputed for them.

Diagnostic provenance. The diagnostic examples were shortlisted using the stored Gemini 3.1 scores and then inspected directly alongside the source, edited proxy, and both output groups at multiple timestamps. They are selected to make an observable synthesis limitation concrete. A proxy is considered informative here when the particular requested edit is visibly present; this does not imply perfect reconstruction of every source detail. Figure 9 shows that the retained bird and mixed snowflake/jellyfish transformation are visible at additional timestamps. The accompanying provenance records pin the scored MP4s, exact decoded frame indices, prompts, conditioning traces, and figure hashes.

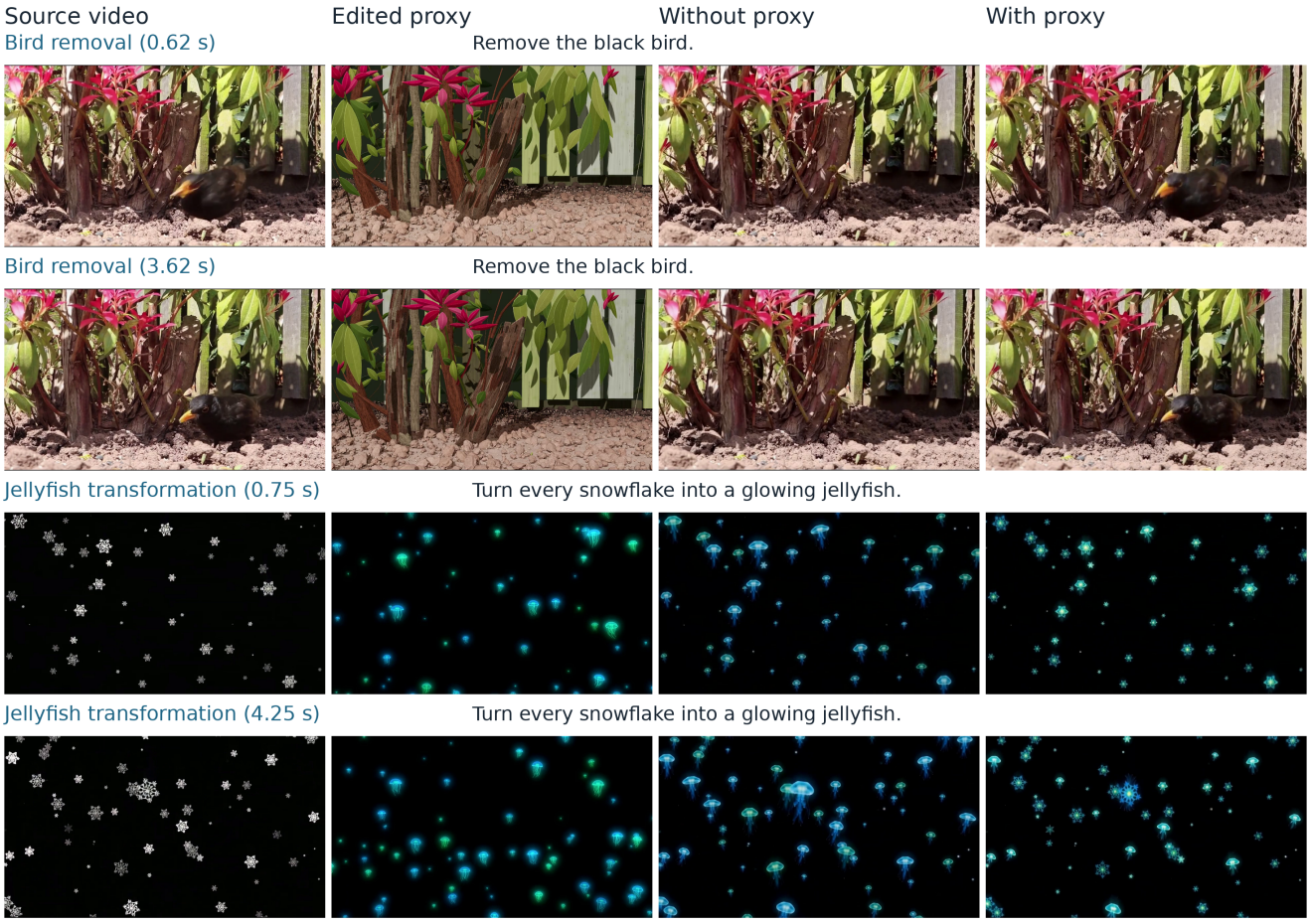

Figure 9. **Additional timestamps for diagnostic cases.** Top pair: the bird remains in the guided video although the proxy omits it. Bottom pair: the guided video mixes retained snowflakes with jellyfish, whereas the proxy depicts the intended transformed forms. All panels preserve the complete normalized composition and use the same videos as Fig. 4.